esiea

ANGULAR

Connexion à une API

Ressources :

https://angular.io/docs

Principe

esiea

- Angular est un framework « pur front ». Il n’est pas prévu pour attaquer directement une base de données ou même des fichiers.
- Sa philosophie est d’être la « partie émergée de l’iceberg », la partie immergée étant une API.
- C’est pourquoi Angular ne propose pas spécialement de drivers permettant l’accès à des BDD mais propose, en contrepartie, le moyen de se connecter à une API.
- C’est ce que propose l’objet **`HttpClient`**.

Pré-requis

esiea

- Pour pouvoir avancer sur ce cours, une API pré-existante doit exister qui doit fournir (au moins) les services suivants :
  - Un service d’authentification ;
  - Un service permettant de créer un système ;
  - Un service renvoyant la liste de tous les systèmes ;
  - Un service renvoyant un système à partir de son identifiant ;
  - Un service permettant de mettre à jour un système ;
  - Un service permettant de supprimer un système ;

Initialisation

esiea

- Pour cette partie du cours, nous allons créer un nouveau projet Angular, puis lui ajouter le module **`http-client`** :

```
ng add http-client
```

- Il faut ensuite ajouter ce module dans les imports du projet (fichier **`app.config.ts`**).

```
export const appConfig: ApplicationConfig = {
  providers: [
      provideBrowserGlobalErrorListeners(),
      provideRouter(routes),
      provideHttpClient()
  ]
};
```

Ajouter cette ligne

Modèle

esiea

- Pour pouvoir travailler avec l'API, nous allons commencer par créer notre modèle (réduit) afin de refléter celui de l'API :

```
ng g e model/UserProfile
ng g cl model/User
ng g cl model/StellarSystem
ng g cl model/Star
ng g cl model/Planet
```

Ces 2 dernières classes seront vides car non utilisées dans ce cours.

```
export enum UserProfile {
    ADMIN="Administrateur",
    WRITER="Rédacteur",
    READER="Lecteur"
}
```

```
export class User {
    id! : number;
    login! : string;
    password! : string;
    profile! : UserProfile;
}
```

```
export class StellarSystem {
    id! : number;
    name! : string;
    posX! : string;
    posY! : string;
    star! : Star;
    planets! : Array<Planet>;
}
```

Services

esiea

- Nous allons aussi créer des services :
  - Une classe **`UsersServices`** qui contient une méthode login recevant login et mot de passe et stockant le token pour une utilisation ultérieure :

```
export class UsersService {
  constructor() { }
  login(login : string, password : string) : void { }
}
```

Toutes les méthodes sont vides pour le moment, même si ça provoque des erreurs.

- Une autre pour les systèmes stellaires :

```
export class StellarSystemsService {
  addSystem(sys : StellarSystem) : StellarSystem {}
  getAllSystems() : Array<StellarSystem> {}
  getSystemById(id : number) : StellarSystem {}
  updateSystem(sys : StellarSystem) : StellarSystem {}
  deleteSystem(id : number) : void {}
  constructor() { }
}
```

Services

esiea

- La dernière classe de service sera celle chargée des échanges avec l'API. Pour cela, on lui injecte, via le constructeur, l'objet **`HttpClient`** :

```
export class APIService {
  constructor(private httpClient: HttpClient) { }
}
```

- L'étape suivante va être de créer dans cet **`APIService`**, les méthodes permettant d'envoyer les différents types de requêtes à l'API.

Gestion des environnements

esiea

- Angular possède un mécanisme permettant de basculer certains paramètres (comme l'URL de l'API) entre le développement et la production de façon transparente pour le développeur.
- On commence par créer les environnements :

```
ng g environments
```

- Cela crée 2 fichiers dans le dossier **`src/environments`** du projet :
  - **`environment.ts`** sera utilisé en production ;
  - **`environment.development.ts`** sera utilisé lors du dev.

Gestion des environnements

esiea

- On remplira ces deux fichiers avec la même constante qu'on valuera ainsi.
  - **`environment.ts`** :

```
export const environment = {
    production: true,
    apiUrl: 'http://my-prod-url/StellarSystem/api'
};
```

  - **`environment.development.ts`** :

```
export const environment = {
    production: false,
    apiUrl: 'http://localhost:8080/StellarSystem/api'
};
```

Gestion des environnements

esiea

- Avec la définition de cette constante, on pourra maintenant définir nos URLs d'accès à nos services API, sans se soucier de l'environnement actuel, en y concaténant les discriminateurs du service voulu.

- Exemple pour la création d'un système :

```
import { environment } from '../../environments/environment';

@Injectable({
  providedIn: 'root'
})
export class StellarSystemsService {
  createUrl : string = environment.apiUrl + "/system/add";
  // Suite de la classe...
```

Importer le fichier de prod.
« `ng serve` » se chargera de remplacer par celui de dev.

Requêtes

esiea

- L'objet **`HttpClient`** possède différentes méthodes pour effectuer des requêtes. Celles qui vont majoritairement nous servir sont les suivantes :
  - **`get<T>(...): Observable<T>`**
  - **`post<T>(...): Observable<T>`**
  - **`patch<T>(...): Observable<T>`**
  - **`put<T>(...): Observable<T>`**
  - **`delete<T>(...): Observable<T>`**
- Toutes ces méthodes sont typées et renvoient un **`Observable`** du même type.

Observable

esiea

- L'objet **`Observable`** suit le design pattern **`Observer`**, il permet, aux clients qui s'y abonnent, d'être notifiés quand quelque chose se passe sur l'élément observé. Exemple :

```
affiche() {
  const mousquetaires = new Observable<string>(observer => {
    observer.next("Athos");
    observer.next("Porthos");
    observer.next("Aramis");
    observer.complete();
  });

  mousquetaires.subscribe({
    next: value => console.log("Le mousquetaire suivant est [%s]", value),
    error: err => console.error(err),
    complete : () => console.log("Plus de mousquetaires !")
  });
}
```

L'`Observable` est typé `string`, ses éléments devront donc être des chaînes de caractères.

On remplit ici la séquence des différents éléments renvoyés.

Notre abonné réagit aux principaux évènements générés par l'observable.

Paramétrage des requêtes

esiea

- Pour les méthodes **`get`** et **`delete`** de **`HttpClient`**, les paramètres sont les suivants :
  - L'URL du service de l'API à atteindre sous la forme d'une chaîne de caractères
  - Les options de la requête sous la forme d'un **`Object`** contenant différentes informations.
- Pour les méthodes **`post`**, **`put`** et **`patch`**, on rajoute, entre ces deux paramètres un objet contenant les données à transmettre à l'API.

Options des requêtes

esiea

- Les options de la requête vont contenir les informations suivantes :
  - Ce que l'on va observer en retour : la réponse complète ou seulement le « **`body`** ».
  - Le type de réponse attendue (« **`text`** » ou « **`json`** » selon les cas)
  - Les entêtes de la requête.
- Ces options se baseront sur la documentation de l'API interrogée, c'est pourquoi il est important que celle-ci soit rédigée rigoureusement.

Requêtes génériques

esiea

- Notre **`APIService`** va contenir une méthode générique par type de requête. Ces méthodes seront typées afin de renvoyer un **`Observable`** de l'objet de modèle attendu :
  - Une méthode pour les **`GET`** qui n'envoie rien via le **`body`**, mais peut avoir des entêtes spécifiques (Token JWT, ...)
  - Une méthode pour les **`POST`**, les **`PUT`** et les **`PATCH`** qui vont toutes envoyer des données via le **`body`** et peuvent avoir des entêtes spécifiques
  - Et enfin, une pour le **`DELETE`** qui n'envoie rien via le **`body`**, mais peut avoir des entêtes spécifiques.

Initialisation APIService

esiea

- Notre classe **`APIService`** va contenir un certain nombre de variables et constantes qui nous seront utiles :

```
export class APIService {

  public static readonly ROOT_URL : String = environment.apiUrl;
  public static readonly DEFAULT_HEADERS : HttpHeaders
      = new HttpHeaders({'Content-Type' : 'application/json'});
  jwt! : string;


  constructor(private httpClient: HttpClient) { }

}
```

Url de la racine de l'API (lue dans l'environnement).

Header par défaut

Variable pour stocker le token JWT apres login.

APIService : gestion des erreurs

esiea

- En cas d'erreur de l'API, on va créer un objet capable de récupérer les erreurs au format défini dans celle-ci :

```
ng g class model/API-error
```

- Si on considère acceptable de récupérer le type de ressource ayant provoqué l'erreur dans une chaîne de caractères, voici son contenu :

```
export class APIError {
   message! : string;
   resourceType! : string;
   id! : number;
}
```

APIService : gestion des erreurs

esiea

- En cas d'erreur lors de l'interrogation de l'API, on appellera la méthode suivante :

```
  httpError(error : HttpErrorResponse) {
    let msg : string = "";
    if (error.error instanceof ErrorEvent) {
      // Client side error
      msg = error.error.message;
    } else {
      // API side error
      const apiError: ApiError | null =
              typeof error.error === 'object' ? error.error as ApiError : null;

      msg =
        "Status : " + error.status + "\n" +
        "Message : " + (apiError?.message ?? error.message) + "\n" +
        "Resource : " + (apiError?.resourceType ?? "unknown") + "\n" +
        "Id : " + (apiError?.id ?? "N/A");
    }
    console.log(msg);
    return throwError(()=>new Error(msg));
  }
```

APIService GET

esiea

- La méthode gérant le GET sera ainsi codée :

```
public sendGetRequest<T>(url : string, headers : HttpHeaders|null)
                        : Observable<T> {
  if (headers == null) {
    headers = APIService.DEFAULT_HEADERS;
}
  let httpOptions : Object = {
    observe : 'body',
    responseType : 'json',
    headers : headers
}
  return this.httpClient.get<T>(url, httpOptions)
                .pipe(retry(1), catchError(this.httpError));
}
```

Typée. Exemple : **sendGetRequest<User>(...)**

Renvoie un Observable sur le type fourni.
Exemple : **Observable<User>**

Si pas de header fourni, utilisation du header par défaut pour GET et DELETE

On s'intéresse uniquement au body de la réponse. Celui-ci doit contenir du Json

En cas d'erreur, on ré-essaie une fois puis, si ça ne marche toujours pas, on appelle la méthode gérant les erreurs.

APIService DELETE

esiea

- La méthode gérant le DELETE sera quasi identique (différences avec le GET encadrées) :

```
  public sendDeleteRequest<T>(url : string, headers : HttpHeaders|null)
                                : Observable<T> {
    if (headers == null) {
      headers = APIService.DEFAULT_HEADERS;
  }
    let httpOptions : Object = {
      observe : 'body',
      responseType : 'json',
      headers : headers
  }
    return this.httpClient.delete<T>(url, httpOptions)
                  .pipe(retry(1), catchError(this.httpError));
}
```

APIService POST, PUT et PATCH

esiea

- Les méthode POST, PATCH et PUT seront codées ainsi :

```
sendPostRequest<T>(url: string,
                   data: T,
                   headers: HttpHeaders | null) : Observable<T> {
  if (headers == null) {
    headers = APIService.FORM_HEADERS;
  }
  let httpOptions: Object = {
    observe: 'response',
    responseType: 'json',
    headers: headers
  }
  return this.httpClient.post<T>(url, data, httpOptions)
                .pipe(retry(1),catchError(this.httpError));
}
```

On reçoit les données à mettre dans le body sous forme d'un objet

Si pas de header fourni, utilisation du header avec formulaire

Pour le POST, on s'intéresse à l'intégralité de la réponse pour pouvoir lire les entêtes lors de l'appel au endpoint de login. Pour les PUT et PATCH, seul le body suffit.

- L'**APIService** ainsi finalisé pourra être réutilisé sans modifications dans tout nouveau projet.

Utilisation de l'APIService

esiea

- Comme tout service, l'**APIService** doit être injecté dans les services fonctionnels pour pouvoir être utilisé.

```
constructor(private apiService: APIService) { }
```

- On va aussi définir des constantes pour spécifier les URL selon les entités visées :

```
readonly ROOT_SYS_URL: String = APIService.ROOT_URL + "/system";
```

Note : le mot-clé **const** n'est pas autorisé dans une classe. On utilise **readonly** à la place.

- Et modifier les méthodes pour qu'elles renvoient des **Observable**.

```
addSystem(sys: StellarSystem): Observable<StellarSystem> {}
getAllSystems(): Observable<Array<StellarSystem>> { }
getSystemById(id: number): Observable<StellarSystem> { }
updateSystem(sys: StellarSystem): Observable<StellarSystem> { }
patchSystem(sys: StellarSystem): Observable<StellarSystem> { }
deleteSystem(id: number): Observable<StellarSystem> { }
```

Le delete aussi renvoie un **Observable** afin d'être sûr que la suppression s'est bien passée

Utilisation de l'APIService

esiea

- Voici les méthodes GET et DELETE pour les systèmes :

```
getAllSystems(): Observable<Array<StellarSystem>> {
  let url = this.ROOT_SYS_URL + "/all"
  return this.apiService.sendGetRequest<Array<StellarSystem>>(url,
null);
}

getSystemById(id: number): Observable<StellarSystem> {
    let url = this.ROOT_SYS_URL + "/" + id;
    return this.apiService.sendGetRequest<StellarSystem>(url, null);
}

deleteSystem(id: number): void {
    let url = this.ROOT_SYS_URL + "/" + id;
    this.apiService.sendDeleteRequest<StellarSystem>(url, null);
}
```

Utilisation de l'APIService

esiea

- Voici les autres méthodes de gestion des systèmes :

```
addSystem(sys: StellarSystem): Observable<StellarSystem> {
  let url = this.ROOT_SYS_URL
  return this.apiService.sendPostRequest<StellarSystem>(url, sys, null);
}

updateSystem(sys: StellarSystem): Observable<StellarSystem> {
  let url = this.ROOT_SYS_URL
  return this.apiService.sendPutRequest<StellarSystem>(url, sys, null);
}

patchSystem(sys: StellarSystem): Observable<StellarSystem> {
  let url = this.ROOT_SYS_URL
  return this.apiService.sendPatchRequest<StellarSystem>(url, sys, null);
}
```

Liste des systèmes

esiea

- Pour tester l'accès à l'API, nous allons créer un composant (qui sera affiché dans le composant principal) :

```
ng g c components/system/systems-list
```

```
export class SystemsListComponent {
  allSystems: Array<StellarSystem>;
  constructor(private service: StellarSystemsService) {
    let obs: Observable<Array<StellarSystem>> = service.getAllSystems();
    this.allSystems = new Array<StellarSystem>();
    obs.subscribe((data: StellarSystem[]) => {
      data.forEach((tmpSys: StellarSystem) => {
        // Copy received data in a new Object
        let curSys: StellarSystem = new StellarSystem(tmpSys.name, tmpSys.posX, tmpSys.posY);
        curSys.id = tmpSys.id;
        curSys.star = tmpSys.star;
        curSys.planets = tmpSys.planets;
        this.allSystems.push(curSys);
      });
    });
  }
}
```

Injection du service fonctionnel.

La copie du système contenu dans l'**Observable** dans une nouvelle instance de **StellarSystem** doit être fait car le premier est un « faux » ne contenant que les attributs mais aucune méthode. Ceci posera des problèmes dès que l'on voudra enrichir notre objet (par des addPlanet(…) et **removePlanet(…)** par exemple).

Liste des systèmes

esiea

- L'affichage (basique) :

```
@for (sys of allSystems; track sys.id) {
    <p>Système "{{sys.name}}" 
            position : ({{sys.posX}},{{sys.posY}})</p>
} @empty {
    <p>la liste est vide.</p>
}
```

- Et son résultat :

Système "Procyon" position : (45,18)

Système "Gama 23" position : (22,137)